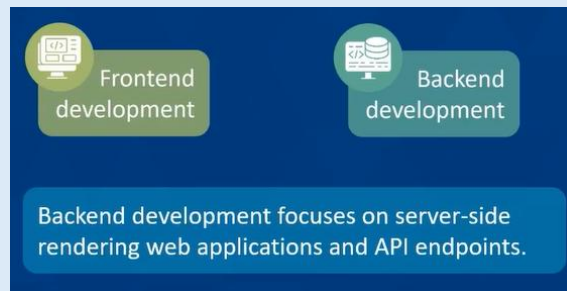


Module 10: Introduction to Backend Development

Module Overview

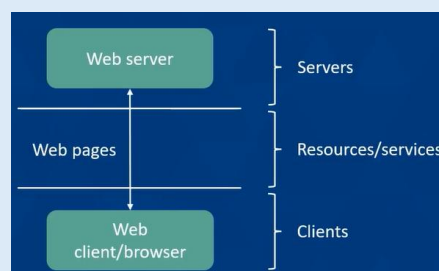
- Machine learning and deep learning models form the foundation of AI systems, but software integration is key to delivering business value.
- Backend development focuses on server-side rendering and API creation, enabling dynamic and scalable web applications for enterprise use.



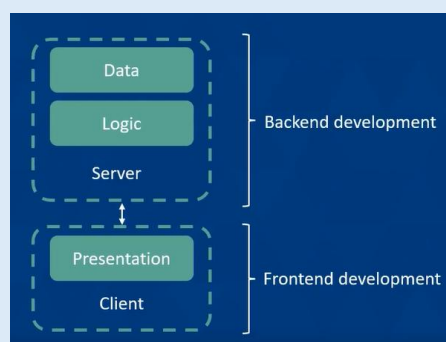
- JavaScript and Python are highlighted as core programming languages, each paired with a suitable framework for backend development.
- Dynamic content generation from databases is a central skill taught, supporting real-time data-driven web page creation and interaction.
- Server-side web development using popular frameworks enables integration of AI models into practical applications that automate decisions and streamline business processes.

Backend Software Engineering with JavaScript and Python

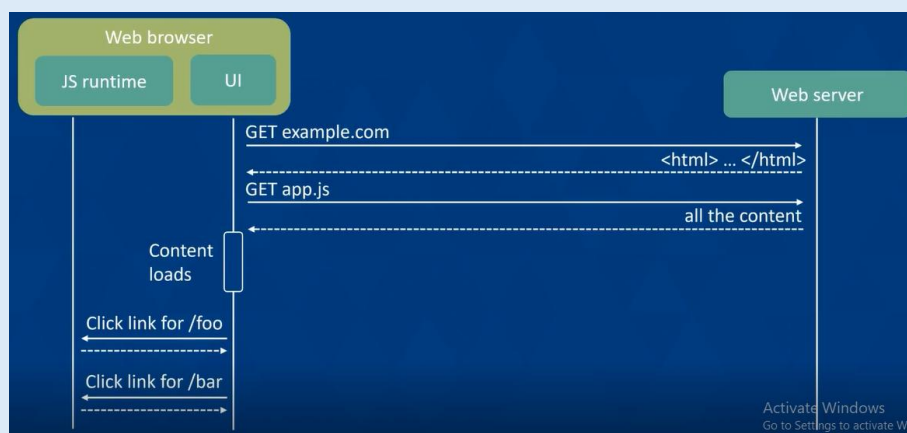
- The differences between frontend and backend development, and between client-side and server-side rendering, are essential to modern web application design.
- Web applications are widely used due to cross-platform compatibility, minimal system requirements, and simplified deployment and maintenance.



- Web applications follow a client-server architecture, where frontend manages presentation and backend handles logic and data processing.



- Server-side rendering sends a new request for each web page, while client-side rendering loads content once and renders pages in-browser.



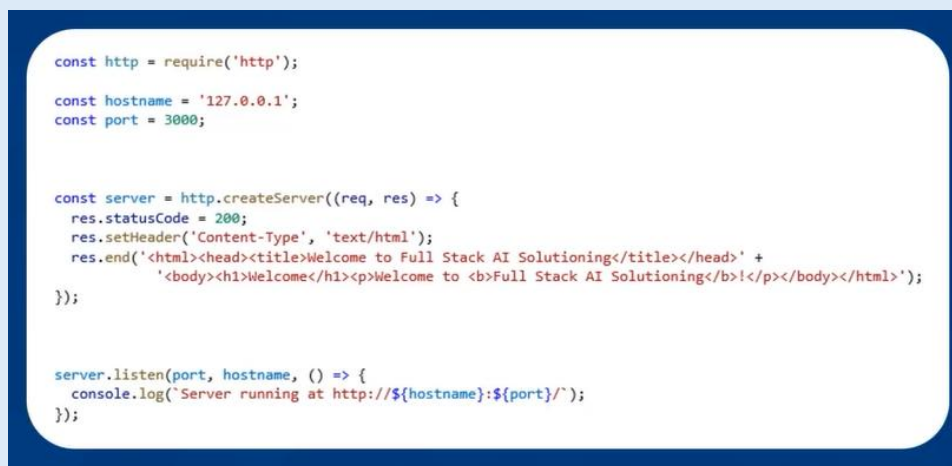
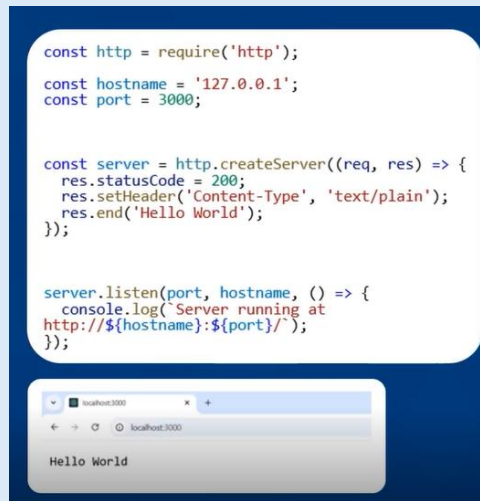
- JavaScript with Express.js and Python with Flask are commonly used to build server-side rendering web applications in modern backend development.

Introduction to Node.js and Express.js

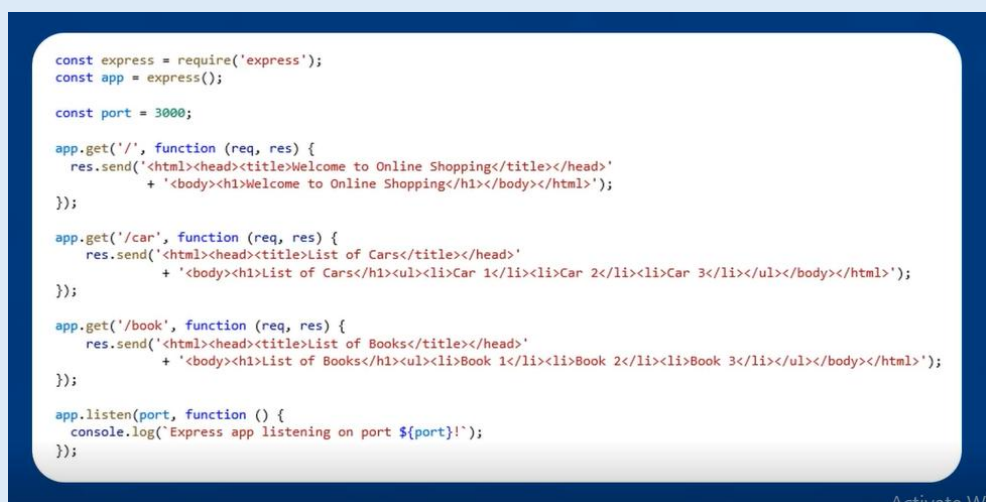
- Node.js extends JavaScript beyond the browser, enabling general-purpose programming and server-side backend development.
- JavaScript code can now run independently of HTML documents, marking a significant shift from traditional client-side scripting.



- Node.js enables basic web server functionality but lacks support for routing, HTTP verb handling, templates, and conversational state management.



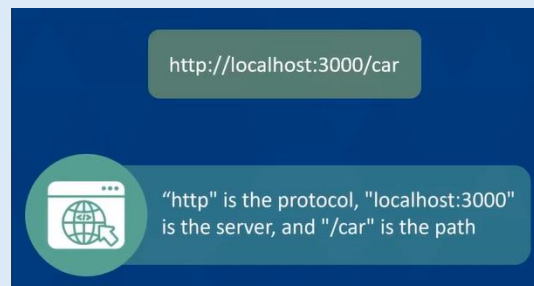
- Express.js, built on top of Node.js, addresses these limitations by offering efficient routing and support for common web development tasks.



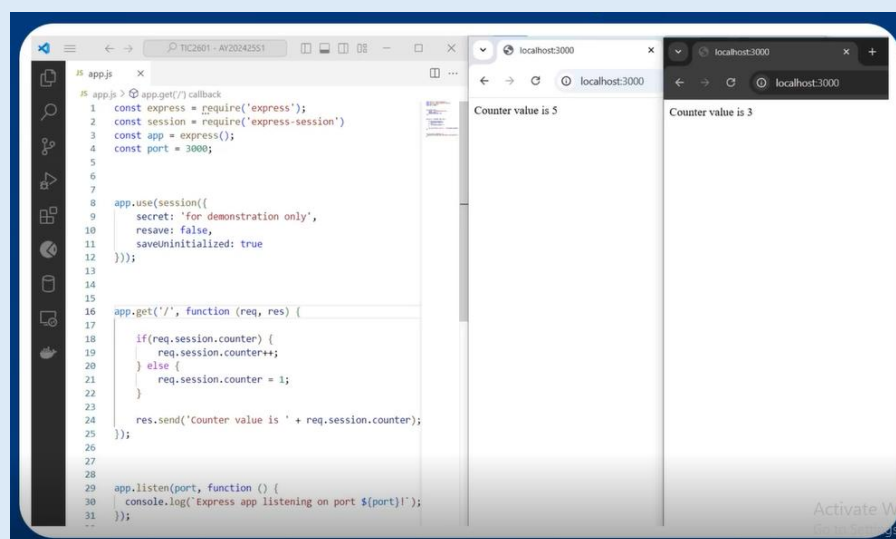
- Express.js is widely adopted and serves as the foundation for many popular Node.js web frameworks used in backend development.

Server-Side Web Application Development with Express.js

- Routing in Express.js maps HTTP methods and paths to handler functions, enabling structured responses to client requests.



- Route parameters and query string parameters allow customisation of content using dynamic values passed within the URI.
- Route parameters support better search engine optimisation compared to query strings, especially in content-heavy web applications.
- Response methods such as `res.send()` and `res.sendFile()` provide flexible ways to return HTML content or files to the client.
- Session middleware in Express.js enables conversational state management by storing user-specific data on the server using session IDs.



Using the Pug Template Engine with Express.js

- Writing HTML directly in JavaScript handler functions becomes complex and inefficient as web content grows in size and complexity.
- Template engines like Pug simplify HTML generation by allowing structured templates to be defined in separate `.pug` files.



Templates store HTML content in separate files.



Pug simplifies HTML syntax using indentation.



Handler functions return templates for browser display.



Pug removes angle brackets and closing tags.



Express.js supports multiple template engines; Pug is the default.



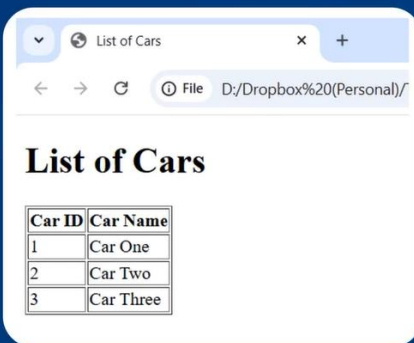
Pug templates use the .pug file extension.

- Pug syntax reduces verbosity by using indentation instead of angle brackets and omitting traditional closing tags.

```

1 <doctype html>
2 <html>
3   <head>
4     <title>List of Cars</title>
5   </head>
6   <body>
7     <h1>List of Cars</h1>
8     <table border="1">
9       <tr>
10        <th>Car ID</th>
11        <th>Car Name</th>
12      </tr>
13      <tr>
14        <td>1</td>
15        <td>Car One</td>
16      </tr>
17      <tr>
18        <td>2</td>
19        <td>Car Two</td>
20      </tr>
21      <tr>
22        <td>3</td>
23        <td>Car Three</td>
24      </tr>
25    </table>
26  </body>
27 </html>

```



- The res.render method in Express.js enables dynamic data to be passed into templates using interpolation with #{} notation.
- Pug supports conditional logic, loops, and the inclusion of static assets, making it suitable for building dynamic and maintainable web pages.

```

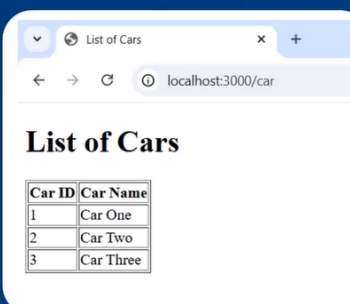
1 const express = require('express');
2 const app = express();
3 const port = 3000;
4
5 app.set('view engine', 'pug');
6
7
8 app.get('/', function (req, res) {
9   res.render('car', { 'showContent': true,
10     'cars': [
11       { 'carId': 1, 'carName': 'Car One' },
12       { 'carId': 2, 'carName': 'Car Two' },
13       { 'carId': 3, 'carName': 'Car Three' }
14     ]
15   });
16
17
18 app.listen(port, function () {
19   console.log('Express app listening on port ' + port);
20 });

```

```

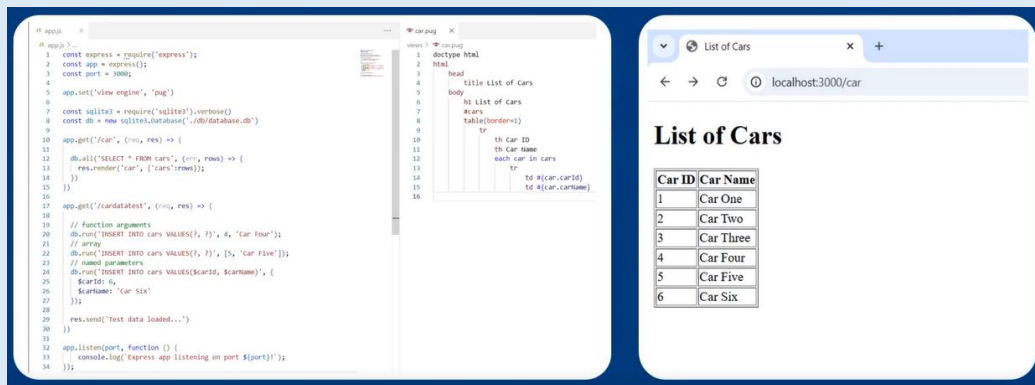
1 doctype html
2 html
3   head
4     title List of Cars
5   body
6     h1 List of Cars
7     #showContent
8     #cars
9     if showContent
10       table(border=1)
11         tr
12           th Car ID
13           th Car Name
14           each car in cars
15             tr
16               td #{car.carId}
17               td #{car.carName}
18         else
19           p Content is not shown

```



Creating Database-Driven Web Application with Express.js

- Integrating a server-side rendering web application with a database enables scalable, dynamic content generation beyond hardcoded data.
- E-commerce platforms benefit from database integration by dynamically displaying product listings and tracking customer orders.
- Express.js allows seamless database connectivity using Node.js drivers and SQL queries for reading and writing data.



- SQLite is a lightweight database used to demonstrate integration, with query results returned as JavaScript objects for easy manipulation.
- Object-relational mapping tools like Sequelize simplify database interactions by managing data models and eliminating the need for raw SQL.

Express.js supports object mapping frameworks like Sequelize

Sequelize:

- Enables working with relational databases using only JavaScript
- Eliminates the need for SQL statements
- Simplifies defining data models and auto-generating database tables

Introduction to Flask

- Flask is a lightweight and beginner-friendly Python framework for building server-side rendering web applications efficiently.

Python offers various web frameworks like Django and Pyramid.

Flask is lightweight, easy to learn, and simple to use.

- Key features of Flask include routing, template rendering with Jinja2, and support for static assets like images and stylesheets.
- Flask enables conversational state management to retain user-specific data across multiple HTTP requests.
- Database integration in Flask is straightforward, with support for SQL and object-relational mapping tools like SQLAlchemy.
- Flask shares core capabilities with other modern web frameworks, making it easier to transition between different development stacks.

Defines multiple routes for different sections, supporting HTTP methods like GET and POST

Formats HTML templates and integrates static assets like images and stylesheets

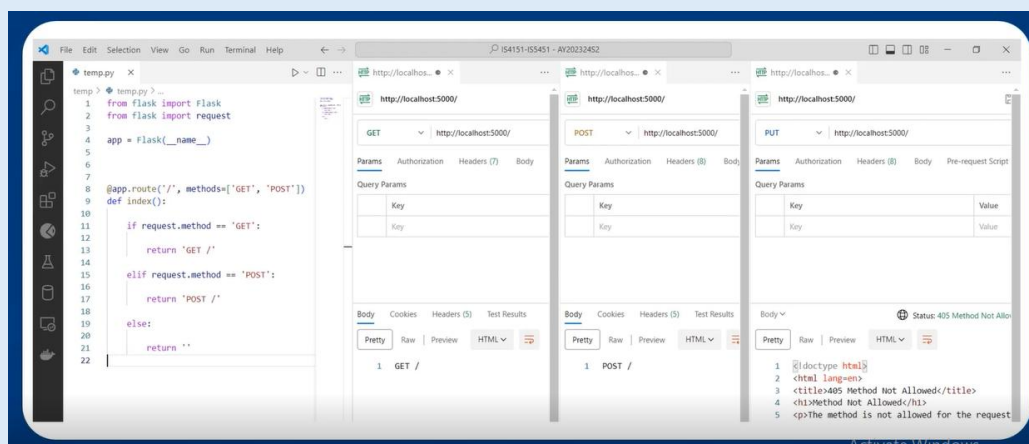
Stores user-specific data across requests

Reads and writes data using SQL or ORM frameworks like SQLAlchemy

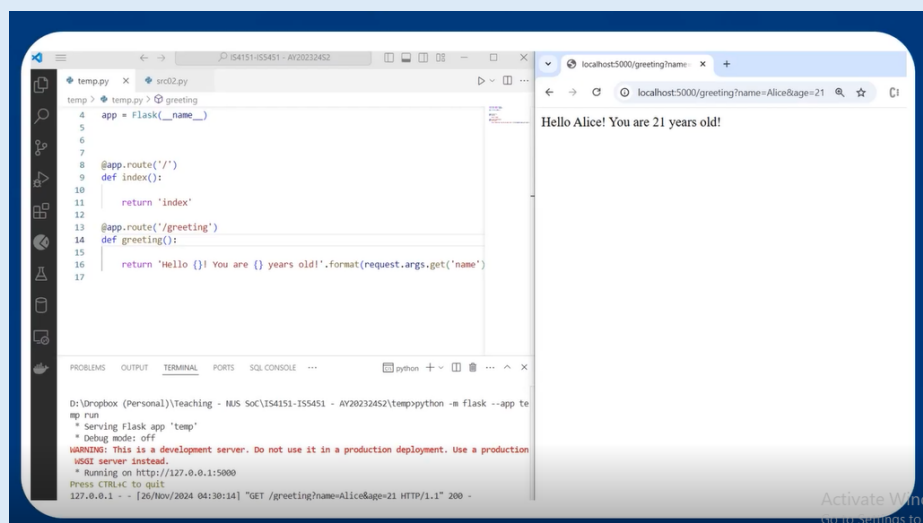
Supports file uploads, cookies, redirections, and error handling

Server-Side Web Application Development with Flask

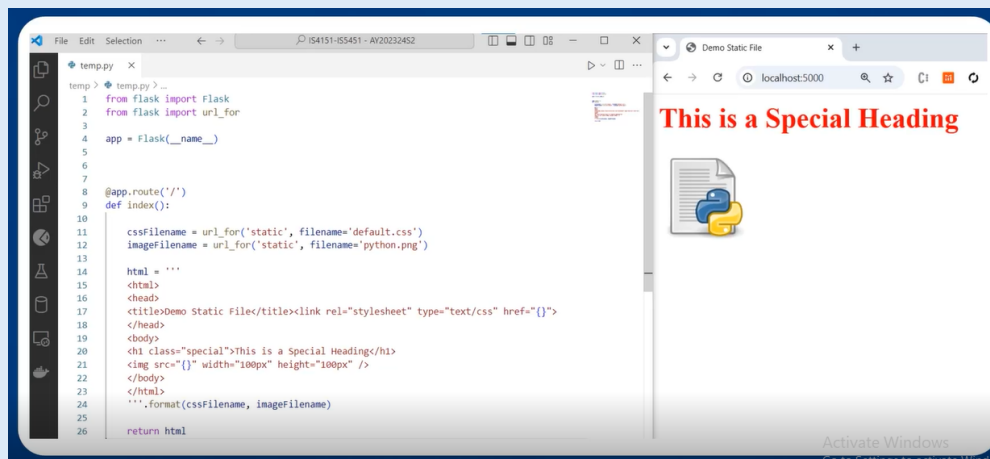
- Flask uses route decorators to bind Python functions to specific paths, returning HTML content to the web browser.
- Multiple HTTP methods like GET and POST can be supported by routes, while unsupported methods return appropriate HTTP error codes.



- Flask supports both path and query string parameters, with optional converters available for automatic type conversion when needed.



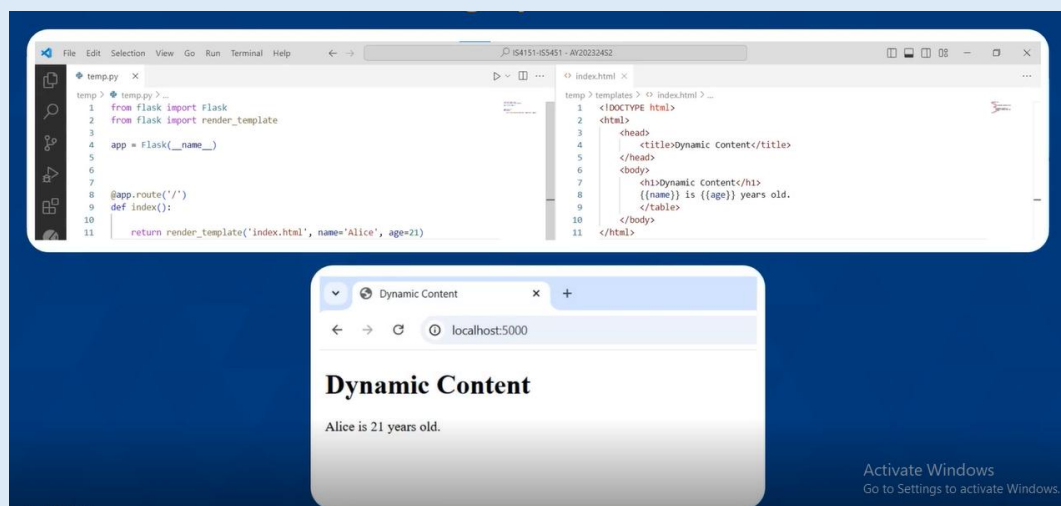
- Static files such as stylesheets and images are served from a dedicated static subfolder and referenced using the `url_for()` function.



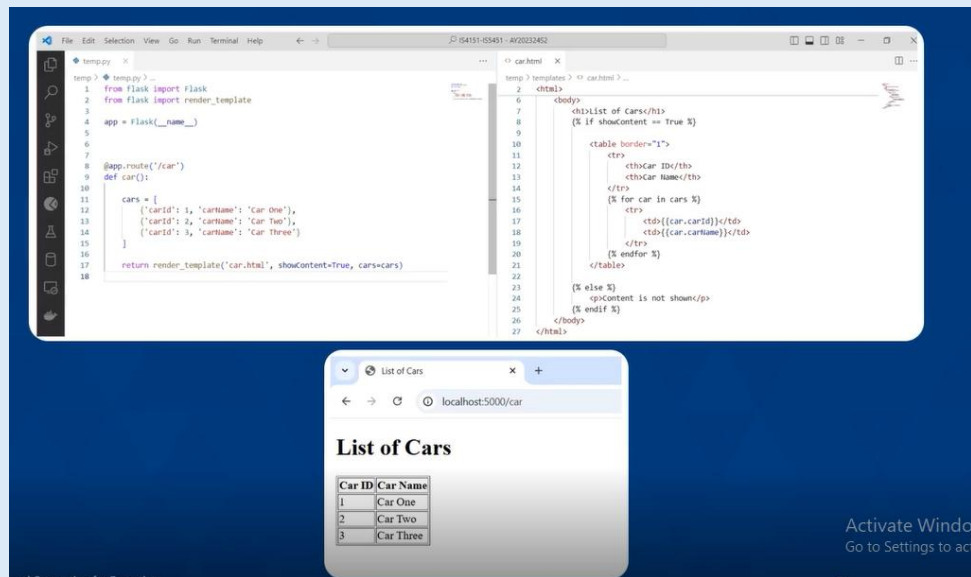
- Conversational state is managed using the session object, allowing user-specific data to persist across multiple HTTP requests.

Using the Jinja2 Template Engine with Flask

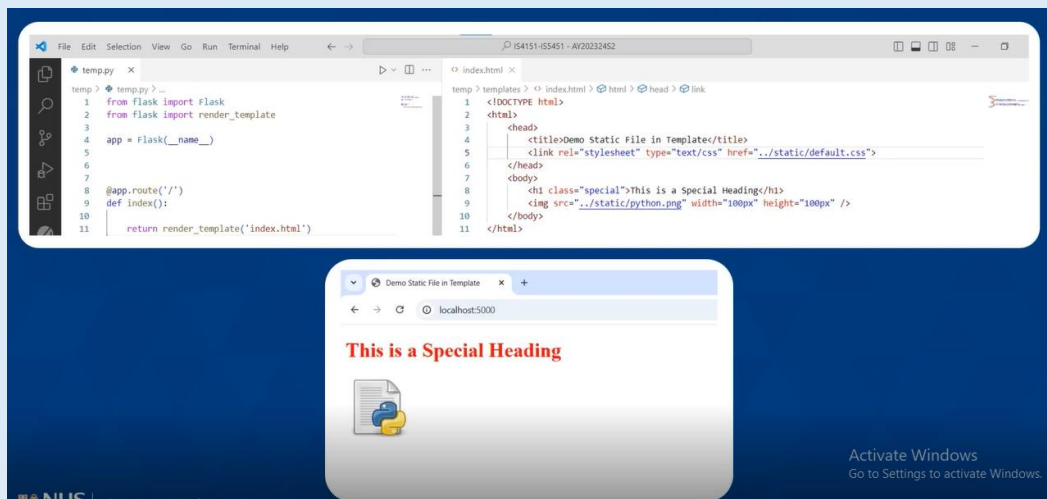
- Flask supports the use of the Jinja2 template engine to separate HTML content from Python functions, making it easier and more intuitive to create structured web pages.
- Jinja2 templates, placed in a templates subfolder, use standard HTML syntax and are rendered using the `render_template()` function, which replaces raw string returns in Python functions.
- Dynamic content is inserted into templates using the `{{ }}` interpolation syntax, allowing variables and expressions to be passed from Python functions into HTML templates.



- Jinja2 supports conditional rendering using if statements and iterative rendering using for loops, which are useful for displaying lists and dictionaries retrieved from code or a database.

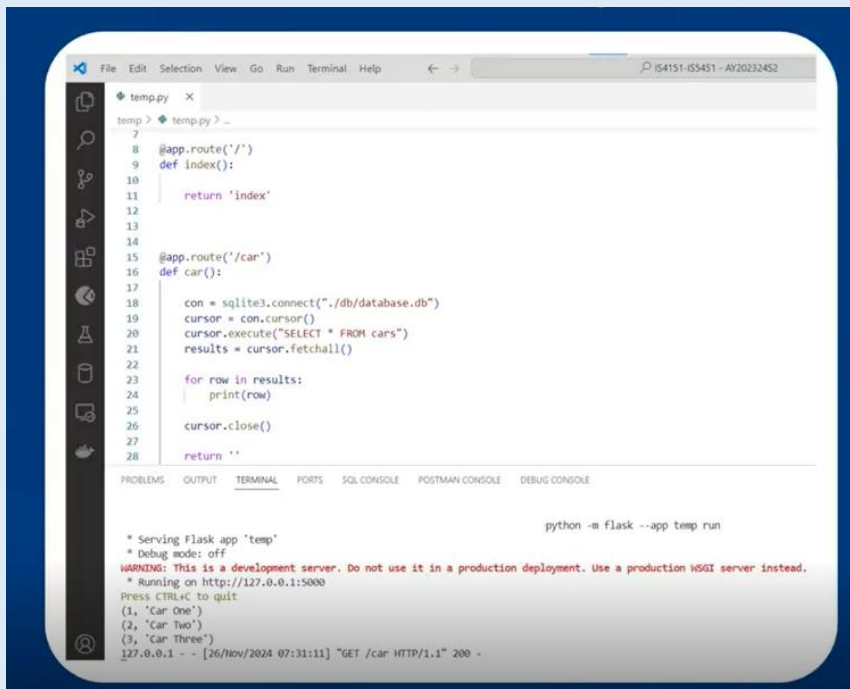


- Static assets such as stylesheets and images can be referenced in Jinja2 templates using relative paths to files stored in the static subfolder, similar to Express.js with Pug.



Creating Database-Driven Web Application with Flask

- Flask integrates seamlessly with relational databases such as SQLite, MySQL, and Postgres using Python's DB-API 2.0 compatible drivers; in this video, SQLite is used for demonstration.
- Database integration involves opening a connection and executing SQL statements through Python's built-in database manipulation functions.



```

temp.py
temp > temp.py > ...
7
8 @app.route('/')
9 def index():
10     return 'index'
11
12
13
14
15 @app.route('/car')
16 def car():
17
18     con = sqlite3.connect("../db/database.db")
19     cursor = con.cursor()
20     cursor.execute("SELECT * FROM cars")
21     results = cursor.fetchall()
22
23     for row in results:
24         print(row)
25
26     cursor.close()
27
28     return ''

```

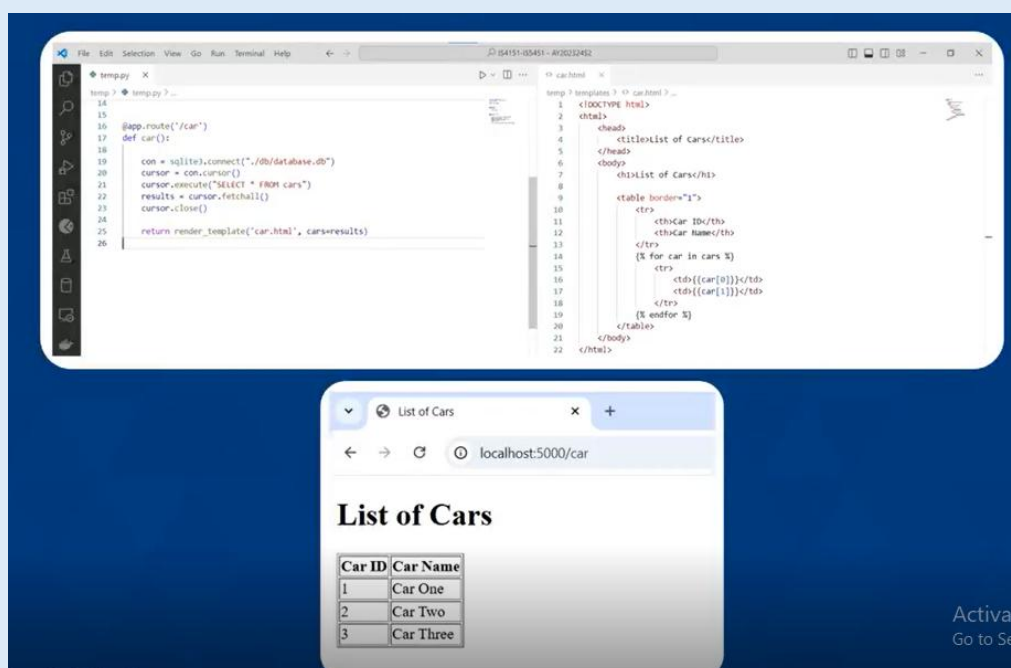
```

python -m flask --app temp run

* Serving Flask app 'temp'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
(1, 'Car One')
(2, 'Car Two')
(3, 'Car Three')
127.0.0.1 - - [26/Nov/2024 07:31:11] "GET /car HTTP/1.1" 200 -

```

- Query results are returned in Python-native structures such as a list of tuples, which can be easily processed and passed into Jinja2 templates for dynamic rendering.



- Data manipulation queries—such as insert, update, and delete—can use placeholders (?) and parameter tuples or string formatting techniques to safely inject values.
- Flask also supports advanced database integration through Object Relational Mapping (ORM) libraries like SQLAlchemy, which allow developers to manage data using Python classes.



Module Summary

- Server-side rendering web applications were developed using Express.js for JavaScript and Flask for Python throughout this module.
- Both frameworks offer strong capabilities for routing, templating, and database integration to support dynamic web page generation.
- Template engines like Pug and Jinja2 simplify HTML creation by separating content structure from application logic and supporting dynamic data.
- Database integration enables the generation of personalised and data-driven content, essential for modern full-stack AI applications.
- Upcoming modules will focus on creating API endpoints to support broader use cases such as mobile apps, IoT, and client-side rendering.



Frameworks Used:
Express.js (JavaScript) and Flask (Python) enable dynamic web page creation using databases and templating.



Full-Stack AI Integration:
These skills are essential for building full-stack AI solutions.



Next Module Focus:
Learn to create API endpoints that return general data instead of HTML content.

THE END